

lz7lbj1kf

February 7, 2025

0.1 API d'Analyse des données de produits récupérées par scraping : visualisation des variations de prix

Importation des bibliothèques nécessaires

```
[1]: from flask import Flask, jsonify, request, send_file, render_template_string
import pandas as pd
import matplotlib.pyplot as plt
from io import BytesIO
from rapidfuzz import fuzz
import seaborn as sns

from difflib import SequenceMatcher
```

Cette section importe les bibliothèques nécessaires pour le développement de l'API web avec Flask, le traitement des données avec pandas, la visualisation avec Matplotlib et Seaborn, ainsi que l'usage de RapidFuzz et difflib pour la comparaison de chaînes de caractères. CORS est également importé pour permettre l'accès à l'API depuis d'autres origines.

Initialisation de l'application Flask avec CORS

```
[2]: from flask_cors import CORS # <-- Import CORS

app = Flask(__name__)
CORS(app)
```

```
[2]: <flask_cors.extension.CORS at 0x1aedad2cfb0>
```

Ici, une instance de l'application Flask est créée et la gestion de CORS (Cross-Origin Resource Sharing) est activée. Cela permet de gérer les requêtes HTTP provenant de domaines différents.

Fonction de normalisation du texte

```
[3]: # Function to normalize text
def normalize_text(text):
    if isinstance(text, str):
        text = text.lower() # Convert to lowercase
        # nom = re.sub(r'[a-z\s]', '', text) # Uncomment if needed for
        ↪special character removal
    return text
```

```

else:
    # Convert non-string inputs to a string or handle them appropriately
    return str(text) if text is not None else ""

```

Cette fonction prend en entrée une chaîne de caractères et la convertit en minuscules pour faciliter la comparaison. Elle gère également les cas où l'entrée n'est pas une chaîne de caractères en la convertissant en texte.

Fonction pour trouver des produits similaires avec RapidFuzz

```

[4]: # Function to find similar noms using RapidFuzz
def find_similar_noms(nom, nom_list, threshold=70):
    similar_noms = []
    for other_nom in nom_list:
        score = fuzz.ratio(nom, other_nom)
        if score >= threshold:
            similar_noms.append((nom, other_nom, score))
    return similar_noms

```

Cette fonction utilise la bibliothèque RapidFuzz pour trouver des chaînes similaires dans une liste. Elle applique la méthode `token_sort_ratio` qui compare les chaînes après avoir trié les mots dans chaque chaîne, et retourne les paires de noms dont la similarité dépasse un certain seuil.

Fonction pour trouver des produits similaires avec difflib

```

[5]: # Function to find similar noms using difflib
def find_similar_noms_difflib(nom, nom_list, threshold=0.7):
    similar_noms = []
    for other_nom in nom_list:
        score = SequenceMatcher(None, nom, other_nom).ratio()
        if score >= threshold:
            similar_noms.append((nom, other_nom, score))
    return similar_noms

```

Cette fonction utilise `difflib.SequenceMatcher` pour comparer deux chaînes de caractères et retourner une mesure de similarité. Si cette mesure est supérieure à un seuil défini, elle renvoie la paire de produits correspondants.

Analyse des promotions par catégorie

```

[6]: # Analyze promotions by category
def promotions_par_categorie(df):
    promotion_df = df[df['promotion'] != ""]
    promotions = (
        promotion_df.groupby(['category', 'promotion'])
        .size()
        .reset_index(name="count")
    )
    return promotions.to_dict(orient='records')

```

Cette fonction analyse les promotions des produits dans le DataFrame, en filtrant les produits ayant une promotion, puis en regroupant les données par catégorie et type de promotion pour compter le nombre de produits dans chaque catégorie.

Analyse du produit avec les plus grandes variations de prix

```
[7]: def analyse_get_top_product_price_variation(df):  
    # Normalize product names  
    #df['normalized_nom'] = df['nom'].apply(normalize_text)  
    #df['normalized_description'] = df['description'].apply(lambda x:   
    ↪ normalize_text(x) if pd.notna(x) else "")  
  
    #df['nom_and_description'] = df['normalized_nom']+" "  
    ↪ "+df['normalized_description']  
    #df['nom_and_description'] = df['normalized_nom']  
  
    # Apply fuzzy matching to group similar product names  
    unique_noms = df['nom_and_description'].unique()  
    nom_groups = {}  
  
    for nom in unique_noms:  
        group = find_similar_noms(nom, unique_noms)  
        for _, other_nom, _ in group:  
            nom_groups[other_nom] = nom # Group similar product names  
  
    # Map the grouped noms back to the DataFrame  
    df['grouped_nom'] = df['nom_and_description'].map(lambda x: nom_groups.  
    ↪ get(x, x))  
  
    # Remove duplicates based on product name and website before counting   
    ↪ occurrences  
    df_unique = df.drop_duplicates(subset=['grouped_nom', 'website'])  
  
    # Count the occurrences of each product by website  
    product_counts = df_unique.groupby(["grouped_nom", "website"]).size().  
    ↪ reset_index(name='count')  
  
    # Find the product with the maximum count across all websites  
    top_product = product_counts.groupby('grouped_nom').agg({'count': 'sum'}).  
    ↪ idxmax().iloc[0]  
  
    # Get the details of that top product across websites  
    top_product_data = product_counts[product_counts['grouped_nom'] ==   
    ↪ top_product]  
  
    # Extract price variations for that product across websites
```

```

    price_variations = df[df['grouped_nom'] == top_product].
↳ drop_duplicates(subset=['website'])[['website', 'prix']]

    return top_product, price_variations.sort_values(by='prix', ascending=True)

```

Cette fonction cherche le produit ayant les plus grandes variations de prix entre les sites en regroupant les noms similaires et en analysant les prix pour ce produit à travers les sites.

Visualisation des variations de prix du produit

```

[8]: def visualisation_plot_price_variations(price_variations, product_name):
    # Visualize price variations by website
    plt.figure(figsize=(10, 6))
    plt.bar(price_variations['website'], price_variations['prix'],
↳ color='skyblue')
    plt.title(f"Price Variations for {product_name} Across Websites")
    plt.xlabel('Website')
    plt.ylabel('Price (USD)')
    #plt.xticks(rotation=45, ha="right")

    img_stream = BytesIO()
    plt.savefig(img_stream, format='png')
    img_stream.seek(0)
    return img_stream

```

Cette fonction génère un graphique des variations de prix d'un produit spécifique à travers différents sites, en utilisant un graphique à barres. Le graphique est ensuite converti en flux d'images pour être renvoyé sous forme de réponse.

Analyse des promotions par categorie

```

[9]: # Analyze data
def analyse_promotions_par_categorie(df):
    # Filter products on promotion
    promotion_df = df[df['promotion'] != ""]

    # Count promotions by category
    promotions_par_categorie = promotion_df.groupby(['category',
↳ 'promotion'])['promotion'].size().reset_index(name="count")

    # Sort by 'count' in descending order and get the top
    top_promotions = promotions_par_categorie.sort_values(by='count',
↳ ascending=False).head(10)

    return top_promotions

```

Visualisation des variations de prix du produit

```
[10]: def visualisation_plot_promotions(promotions_par_categorie, title="Promotion_
↳Count by Category and Promotion Type", xlabel="Category", ylabel="Promotion_
↳Count"):
    # Create a Seaborn barplot for better visual representation
    plt.figure(figsize=(12, 6))
    sns.barplot(data=promotions_par_categorie.sort_values(by='count',
↳ascending=True),
                x='category',
                y='count',
                hue='promotion',
                palette="Set2")

    # Customize the plot
    plt.title(title)
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)
    plt.xticks(rotation=45, ha="right")

    img_stream = BytesIO()
    plt.savefig(img_stream, format='png')
    img_stream.seek(0)
    return img_stream
```

Ce code définit une fonction d'analyse des produits par site et date :

analyse_group_by_nom_website_date : Agrège les prix par produit, site et date, filtre les lignes où le prix minimum est égal au prix maximum, trie les résultats par le nombre d'occurrences, puis retourne les produits les plus fréquents.

```
[11]: # Function to group by 'nom', 'website', and 'date_scraped', and sort by count
def analyse_group_by_nom_website_date(df):
    # Group by 'nom' and 'website' and calculate min, mean, max, and count of
↳'prix'
    avg_prices = df.groupby(["nom", "website"])["prix"].agg(["min", "mean",
↳"max", "count"])

    # Filter out rows where the min price is equal to the max price
    grouped = avg_prices[avg_prices["min"] != avg_prices["max"]]

    # Sort by 'count' in descending order to get the top products
    grouped_sorted = grouped.sort_values(by='count', ascending=False)

    # Get the top products with the most occurrences
    products_by_date = grouped_sorted.groupby('nom').head(1).
↳sort_values(by='count', ascending=False).head(10)

    # Reset the index so 'nom' becomes a column again
    products_by_date = products_by_date.reset_index()
```

```
return products_by_date
```

Ce code définit une fonction de visualisation des variations de prix au fil du temps pour les produits sélectionnés :

visualisation_plot_price_variations_by_date : Affiche un graphique en ligne des variations de prix par date pour les produits sélectionnés, avec des annotations pour les promotions et en supprimant les minutes et secondes de la date pour une meilleure lisibilité.

```
[12]: # Function to plot price variations by 'date_scraped' for the top products
def visualisation_plot_price_variations_by_date(df, products_by_date):
    # Filter the original dataframe to include only the top products
    filtered_df = df[df['nom'].isin(products_by_date['nom'])].copy()

    # Replace NaN values in the 'promotion' column with 'No promo' using .loc
    filtered_df.loc[:, 'promotion'] = filtered_df['promotion'].fillna("")

    # Convert 'date_scraped' to datetime if not already
    filtered_df['date_scraped'] = pd.to_datetime(filtered_df['date_scraped'])

    # Remove minutes and seconds from the date
    filtered_df['date_scraped'] = filtered_df['date_scraped'].dt.date

    # Create the Seaborn plot showing price variation by date
    plt.figure(figsize=(12, 6))
    sns.lineplot(data=filtered_df, x='date_scraped', y='prix', hue='nom',
↪marker='o')

    # Add promotion names as annotations on the plot
    for _, row in filtered_df.iterrows():
        plt.text(row['date_scraped'], row['prix'], row['promotion'],
                color='black', fontsize=9, ha='center', va='bottom')

    # Customize the plot
    plt.title("Price Variation by Date for Top Products")
    plt.xlabel("Date Scraped")
    plt.ylabel("Price (USD)")
    plt.xticks(rotation=45, ha="right")
    img_stream = BytesIO()
    plt.savefig(img_stream, format='png')
    img_stream.seek(0)
    return img_stream
```

Ce code définit une fonction d'analyse des données de prix par site :

analyse_data_in_same_site_grouped_sites : Agrège les prix par produit et par site, filtre les lignes où les prix minimum et maximum sont identiques, puis calcule les prix moyens, minimum et maximum par site avant de renvoyer les résultats triés par prix moyen.

```
[13]: # Analyze data
def analyse_data_in_same_site_grouped_sites(df, nom=None, website=None):
    # Group by 'nom' and 'website' to get the average prices and count
    avg_prices = df.groupby(["nom", "website"])["prix"].agg(["min", "mean", "max", "count"])

    # Filter out rows where min equals max
    avg_prices_filtered = avg_prices[avg_prices["min"] != avg_prices["max"]]

    # Group by 'website' and calculate mean of 'min', 'mean', and 'max'
    grouped_by_date = avg_prices.groupby(["website"]).agg({
        'min': 'min',
        'mean': 'mean',
        'max': 'max'
    }).reset_index()

    # Return the result sorted by 'min' in ascending order
    return grouped_by_date.sort_values(by='min', ascending=True)
```

Ce code définit une fonction de visualisation des données de prix moyens par produit :
visualisation_plot_data : Affiche un graphique à barres des prix moyens par produit avec des options de personnalisation pour les étiquettes des axes, et les colonnes utilisées pour les données.

```
[14]: # Plot data
def visualisation_plot_data(avg_prices, nom="Average Prices by Product", xlabel="Product", ylabel="Price (USD)", x=None, y=None):
    # Ensure the DataFrame is indexed properly for plotting
    avg_prices = avg_prices.reset_index() # Reset index for clean plotting

    # Plot the data
    avg_prices.plot(kind="bar", title=nom, xlabel=xlabel, ylabel=ylabel, x=x, y=y)

    # Adjust layout for better display
    plt.xticks(rotation=45, ha="right")

    img_stream = BytesIO()
    plt.savefig(img_stream, format='png')
    img_stream.seek(0)
    return img_stream
```

Point d'entrée Flask pour rechercher des produits similaires

```
[15]: @app.route('/search_similar_products', methods=['GET'])
def search_similar_products():
    # Parse input JSON
    product_name = request.args.get('product_name', '').lower()
```

```

if not product_name:
    return jsonify({"error": "Product name is required"}), 400

# Load dataset
try:
    df = pd.read_csv("Electromenagerscleaned_data.csv")
except FileNotFoundError:
    return jsonify({"error": "Dataset not found"}), 500

# Normalize noms in the dataset
df['normalized_nom'] = df['nom'].apply(normalize_text)

# Find similar noms using difflib
similar_noms = find_similar_noms_diffliib(product_name, df['normalized_nom'].
↪unique())

if not similar_noms:
    return jsonify({"message": "No similar products found"}), 404

# Filter the dataframe for matching noms
matching_noms = [nom[1] for nom in similar_noms]
filtered_df = df[df['normalized_nom'].isin(matching_noms)]

# Group results by nom and website, preserving original indexes
grouped_results = (
    filtered_df.groupby(["nom", "website"], as_index=False)
    .agg(
        mean_price=("prix", "mean"),
        min_price=("prix", "min"),
        max_price=("prix", "max"),
        count=("prix", "count"),
        original_indexes=("normalized_nom", lambda x: (filtered_df.loc[x.
↪index].index + 2).tolist())
    )
    .sort_values(by="min_price")

# Convert results to dictionary
result_data = grouped_results.to_dict(orient='records')

#return jsonify({
#    "input_product": product_name,
#    "similar_products": result_data
#})

return jsonify(result_data)

```


Cette route API permet de rechercher des produits similaires en fonction du nom du produit passé en paramètre. Elle charge un dataset, applique des méthodes de normalisation et de comparaison, puis retourne les produits similaires trouvés.

```
[16]: @app.route('/all_promotions_par_categorie', methods=['GET'])
def analyze_promotions():
    df = pd.read_csv("Electromenagerscleaned_data.csv")
    analysis_results = promotions_par_categorie(df)
    return jsonify(analysis_results)
```

Ce code définit deux routes Flask :

- **/analyse_top_product** : Retourne les données du produit le plus populaire et ses variations de prix sous forme de JSON.
- **/plot_analyse_top_product** : Génère et renvoie une visualisation des variations de prix du produit le plus populaire sous forme d'image.

```
[17]: @app.route('/analyse_top_product', methods=['GET'])
def api_analyse_top_product():
    df = pd.read_csv("Electromenagerscleaned_data.csv")
    top_product, price_variations = analyse_get_top_product_price_variation(df)
    return jsonify({
        'top_product': top_product,
        'price_variations': price_variations.to_dict(orient='records')
    })

@app.route('/plot_analyse_top_product', methods=['GET'])
def plot_analyse_top_product():
    df = pd.read_csv("Electromenagerscleaned_data.csv")
    top_product, price_variations = analyse_get_top_product_price_variation(df)
    img_stream = visualisation_plot_price_variations(price_variations,
    ↪top_product)
    return send_file(img_stream, mimetype='image/png')
```

Ce code définit deux routes Flask :

- **/analyse_promotions** : Retourne les données agrégées des promotions par catégorie sous forme de JSON.
- **/plot_analyse_promotions** : Génère et renvoie une visualisation du nombre de promotions par catégorie et type de promotion sous forme d'image.

```
[18]: @app.route('/analyse_promotions', methods=['GET'])
def api_analyse_promotions():
    df = pd.read_csv("Electromenagerscleaned_data.csv")
    top_promotions = analyse_promotions_par_categorie(df)
    return jsonify({'top_promotions': top_promotions.to_dict(orient='records')})

@app.route('/plot_analyse_promotions', methods=['GET'])
def plot_analyse_promotions():
    df = pd.read_csv("Electromenagerscleaned_data.csv")
```

```

    img_stream = ␣
    ↪visualisation_plot_promotions(analyse_promotions_par_categorie(df),
        title="Promotion Count by Category and Promotion Type",
        xlabel="Category",
        ylabel="Promotion Count")
    return send_file(img_stream, mimetype='image/png')

```

Ce code définit deux routes Flask :

- `/products_by_date` : Retourne les données agrégées des produits par date sous forme de JSON.
- `/plot_products_by_date` : Génère et renvoie une visualisation des variations de prix des produits au fil du temps sous forme d'image.

```

[19]: @app.route('/products_by_date', methods=['GET'])
def products_by_date():
    df = pd.read_csv("Electromenagerscleaned_data.csv")
    products_by_date = analyse_group_by_nom_website_date(df_cleaned)
    return jsonify({'products_by_date': products_by_date.
    ↪to_dict(orient='records')})

@app.route('/plot_products_by_date', methods=['GET'])
def plot_products_by_date():
    df = pd.read_csv("Electromenagerscleaned_data.csv")
    img_stream = visualisation_plot_price_variations_by_date(df, ␣
    ↪analyse_group_by_nom_website_date(df))
    return send_file(img_stream, mimetype='image/png')

```

Ce code définit deux routes Flask :

- `/site_grouped_sites` : Retourne les données agrégées des prix par site sous forme de JSON.
- `/plot_site_grouped_sites` : Génère et renvoie une visualisation des variations de prix par site sous forme d'image.

```

[20]: @app.route('/site_grouped_sites', methods=['GET'])
def site_grouped_sites():
    df = pd.read_csv("Electromenagerscleaned_data.csv")
    site_grouped_sites = analyse_data_in_same_site_grouped_sites(df)
    return jsonify({'site_grouped_sites': site_grouped_sites.
    ↪to_dict(orient='records')})

@app.route('/plot_site_grouped_sites', methods=['GET'])
def plot_site_grouped_sites():
    df = pd.read_csv("Electromenagerscleaned_data.csv")
    img_stream = ␣
    ↪visualisation_plot_data(analyse_data_in_same_site_grouped_sites(df), "Prices ␣
    ↪by sites", "Product", "Price (USD)", "website", ["min", "mean", "max"])
    return send_file(img_stream, mimetype='image/png')

```

Ce code définit une route pour la page d'accueil dans l'API

```

[21]: @app.route('/')
def home():
    return render_template_string("""
        <!DOCTYPE html>
        <html lang="en">
        <head>
            <meta charset="UTF-8">
            <meta name="viewport" content="width=device-width, initial-scale=1.
↪0">

            <title>Data Analysis Web App</title>
            <style>
                body { font-family: Arial, sans-serif; }
                .container { width: 80%; margin: 0 auto; padding: 20px; }
                .button { margin: 10px 0; padding: 10px 20px; background-color:
↪#4CAF50; color: white; border: none; cursor: pointer; }
                .results { margin-top: 20px; }
                pre { background-color: #f4f4f4; padding: 10px; }
                .plot { margin-top: 20px; text-align: center; }
                #loader { display: none; }
                .result-item { margin: 10px 0; padding: 10px; border: 1px solid
↪#ddd; background-color: #f9f9f9; }
                .result-item h3 { margin: 0; }
                .result-item .key { font-weight: bold; }
                .result-item .value { margin-left: 10px; }
                .search-bar { margin-right: 10px; }
            </style>
        </head>
        <body>

            <div class="container">
                <h1>Products Analysis Web App</h1>

                <!-- Search Bar and Button -->
                <input type="text" id="searchInput" class="search-bar"
↪placeholder="Search Similar Products">
                <button class="button" onclick="searchSimilarProducts()">Search
↪Similar Products</button>
                <button class="button"
↪onclick="getData('analyse_promotions')">Analyze Promotions by Category</
↪button>
                <button class="button"
↪onclick="getData('analyse_top_product')">Analyze Product by Different Sites</
↪button>
                <button class="button"
↪onclick="getData('site_grouped_sites')">Analyze Products in Same Site by
↪Date</button>
    """)

```

```

        <button class="button"
        ↪onclick="getData('site_grouped_sites')">Analyze Prices by All Sites</button>

        <br />
        <button class="button"
        ↪onclick="generatePlot('plot_analyse_promotions')">Visualize Promotions by
        ↪Category</button>
        <button class="button"
        ↪onclick="generatePlot('plot_analyse_top_product')">Visualize Product by
        ↪Different Sites</button>
        <button class="button"
        ↪onclick="generatePlot('plot_products_by_date')">Visualize Products in Same
        ↪Site by Date</button>
        <button class="button"
        ↪onclick="generatePlot('plot_site_grouped_sites')">Visualize Prices by All
        ↪Sites</button>

        <div id="loader">Loading...</div>
        <div class="plot" id="plotContainer"></div>

        <div class="results">
            <h2>Results</h2>
            <div id="resultOutput"></div>
        </div>

    </div>

    <script>
        const apiBaseUrl = 'http://localhost:5000/'; // Assuming your
        ↪Flask server is running locally

        // Function to show loader
        function showLoader() {
            document.getElementById('loader').style.display = 'block';
        }

        // Function to hide loader
        function hideLoader() {
            document.getElementById('loader').style.display = 'none';
        }

        // Function to call Flask API and display results
        function getData(endpoint) {
            showLoader(); // Show loader before the request

            fetch(apiBaseUrl + endpoint)

```

```

        .then(response => response.json())
        .then(data => {
            hideLoader(); // Hide loader once data is fetched
            displayData(data); // Function to display data in
↪divs
        })
        .catch(error => {
            hideLoader(); // Hide loader in case of an error
            document.getElementById('resultOutput').innerHTML =
↪`Error: ${error}`;
        });
    }

    // Function to handle Search Similar Products request with
↪search query
    function searchSimilarProducts() {
        const query = document.getElementById('searchInput').value;
↪ // Get the search query

        if (query) {
            showLoader(); // Show loader before the request

            fetch(apiBaseUrl + 'search_similar_products?
↪product_name=' + encodeURIComponent(query))
                .then(response => response.json())
                .then(data => {
                    hideLoader(); // Hide loader once data is
↪fetched
                    displayData(data); // Function to display data
↪in divs
                })
                .catch(error => {
                    hideLoader(); // Hide loader in case of an
↪error
                    document.getElementById('resultOutput').
↪innerHTML = `Error: ${error}`;
                });
        } else {
            alert('Please enter a search term.');
```

```

        }
    }

    // Function to display JSON data in divs
    function displayData(data) {
        const resultDiv = document.getElementById('resultOutput');
        resultDiv.innerHTML = ''; // Clear previous results
    }
}

```

```

// Recursive function to display data as divs
function createDivs(item, parentDiv) {
  if (Array.isArray(item)) {
    item.forEach((subItem, index) => {
      const div = document.createElement('div');
      div.className = 'result-item';
      div.innerHTML = `<h3>Item ${index + 1}</h3>`;
      createDivs(subItem, div); // Recursively handle
↳array elements

      parentDiv.appendChild(div);
    });
  } else if (typeof item === 'object' && item !== null) {
    Object.keys(item).forEach(key => {
      const div = document.createElement('div');
      div.className = 'key-value-pair';
      div.innerHTML = `<span class="key">${key}</
↳span>`;

      const valueDiv = document.createElement('div');
      valueDiv.className = 'value';

      // Recursively handle nested objects or arrays
      createDivs(item[key], valueDiv);

      div.appendChild(valueDiv);
      parentDiv.appendChild(div);
    });
  } else {
    const div = document.createElement('div');
    div.className = 'value';
    div.innerHTML = `${item}`; // Display the value if
↳it's a simple data type

    parentDiv.appendChild(div);
  }
}

// Call the recursive function on the data
createDivs(data, resultDiv);
}

// Function to generate plot
function generatePlot(endpoint) {

  showLoader(); // Show loader before the request

  fetch(apiBaseUrl + endpoint)

```

```

        .then(response => response.blob())
        .then(imageBlob => {
            hideLoader(); // Hide loader once the plot is
↳generated
            const imageUrl = URL.createObjectURL(imageBlob);
            document.getElementById('resultOutput').innerHTML =
↳``;
        })
        .catch(error => {
            hideLoader(); // Hide loader in case of an error
            document.getElementById('resultOutput').innerHTML =
↳`Error generating plot: ${error}`;
        });

    }
    </script>

</body>
</html>
""")

```

Exécution principale : Chargement des données, et analyse des variations de prix des produits

```

[ ]: if __name__ == "__main__":
    #app.run(debug=True)
    app.run(use_reloader=False)

```

```

* Serving Flask app '__main__'
* Debug mode: off

```

WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.

```

* Running on http://127.0.0.1:5000

```

Press CTRL+C to quit

```

127.0.0.1 - - [05/Feb/2025 01:03:47] "GET / HTTP/1.1" 200 -

```

```

127.0.0.1 - - [05/Feb/2025 01:03:50] "GET /analyse_promotions HTTP/1.1" 200 -

```